Activity 2.1: Fast API Instilization and import libraries

- Import all required libraries for the project

```
import os

import json

import re

import base64

import shutil

import urllib.parse

from fastapi import FastAPI, HTTPException, Depends, File, UploadFile, Form, Request, status, Cookie

from fastapi.responses import JSONResponse, RedirectResponse

from fastapi.middleware.cors import CORSMiddleware

from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm

from fastapi.staticfiles import StaticFiles

from fastapi.templating import Jinja2Templates

from fastapi.responses import HTMLResponse

from typing import List, Optional, Dict, Any, Union

from pydantic import BaseModel, EmailStr

from passlib.context import CryptContext

from jose import JWTError, jwt

from datetime import datetime, timedelta

from PIL import Image

from io import BytesIO

import google.generativeai as genai
```

```
from dotenv import load_dotenv

import asyncio

from typing import Dict, Any, List, Optional

import uuid
```

- Fast API Initialization

```
#FastAPI app initialization

app=FastAPI(title=”PocketSmart: AI budget Planner”)

SECRET-KEY=os.getenv(“SECRET-KEY”,”your-secret-key”)

ALGORITHM=”HS256”

ACCESS-TOKEN-EXPIRE-MINUTES=30
```

- Load .env environment

```
# Load environment variables

load_dotenv()


# Configure Google Gemini AI - Do this once, with consistent environment variable name

API_KEY = os.getenv("GOOGLE_API_KEY")

if not API_KEY:

   # Try alternative environment variable name

   API_KEY = os.getenv("GEMINI_API_KEY")

if not API_KEY:

   raise ValueError("No Google API key found in environment variables. Please set
GOOGLE_API_KEY in your .env file.")

genai.configure(api_key=API_KEY)
```

```
# Password hashing context

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token", auto_error=False) # Allow optional token

class RegisterUser(BaseModel):

    username: str

    email: str

    full_name: Optional[str] = None

    password: str

# Configure CORS

app.add_middleware(

    CORSMiddleware,

    allow_origins=["*"], # Adjust in production

    allow_credentials=True,

    allow_methods=["*"],

    allow_headers=["*"],

)

# Static files and templates

templates = Jinja2Templates(directory="templates")

os.makedirs("static/uploads", exist_ok=True)

app.mount("/static", StaticFiles(directory="static"), name="static")
```